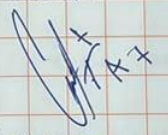
Modelo de programación funcional

El modelo de programación funcional es un paradigma de programación que se basa en el uso de funciones como elemento principal para resolver problemas. A diferencia de la programación imperativa, donde se indica paso a paso cómo debe realizarse una tarea y se modifican variables, en la programación funcional se busca describir qué resultado se debe obtener mediante la composición y aplicación de funciones.

Una característica importante es que intenta trabajar con datos inmutables, es decir, datos que no se modifican después de haber sido creados. Las funciones reciben valores de entrada y producen valores de salida, procurando evitar cambios de estado y efectos secundarios.



Características principales

1. Funciones = Son la parte fundamental del programa
2. Inmutabilidad = Los datos no se modifican una vez creados
3. Funciones puras = Para una misma entrada producen siempre el mismo resultado
4. Composición de funciones = Varias funciones pueden combinarse para resolver problemas más complejos.
5. Recursividad = Se utilica frecuentemente en lugar de estructuras repetitivas como algunos ciclos.
6. Funciones de orden superior = Una función puede recibir otra función como argumento o devolver una función como resultado.
7. Menor dependencia del estado = Se intenta reducir la modificación de variables y estados globales.

Ejemplo

```
dobles = map (*2) [1,2,3,4,5]
Resultado = [2,4,6,8,10]
```

← Aquí, map aplica la función (*2) a cada elemento de la lista.

Nombre: Yanelly Cruz Nicolás

Lenguajes relacionados

Algunos lenguajes que utilizan programación funcional son Haskell, Lisp, Scheme, Clojure, Erlang, F# y OCaml. También lenguajes multiparadigma como Python, JavaScript, Java, C++ incorporan características de programación funcional.

En inteligencia artificial

La programación funcional puede ser útil inteligencia artificial porque permite trabajar de manera estructurada con conjuntos de datos, transformar información mediante funciones y construir algoritmos mediante la combinación de operaciones pequeñas.

Nombre: Yanelly Cruz Nicolás

Cálculo Lambda de Alonzo Church

¿Qué es el cálculo Lambda?

El cálculo Lambda (λ - calculus) es un sistema formal creado por el matemático y lógico Alonzo Church en la década de 1930. Es uno de los fundamentos teóricos de la programación funcional y permite representar la definición y aplicación de funciones.

Se basa principalmente en tres conceptos:

1. Abstracción = Consiste en definir una función
   $\lambda x . x + 1$

Significa = Una función que recibe x y devuelve $x + 1$

2. Aplicación = Consiste en utilizar una función con un valor
   $(\lambda x . x + 1)\ 5 \longrightarrow 6$

3. Reducción = Es el proceso de evaluar una expresión para obtener su resultado

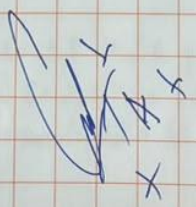
Tenemos → $\lambda x . x * 2$

Esta expresión representa una función que recibe un número y los multiplica por 2.

si aplicamos la función a 4

$(\lambda x . x * 2)\ 4 \longrightarrow 4 * 2 \longrightarrow 8$

El cálculo lambda es importante porque proporciona una base matemática para entender cómo funcionan las funciones en la programación funcional. Lenguajes como Haskell, Lisp y Scheme están fuertemente relacionados con estos conceptos.

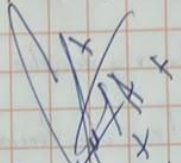


Nombre: Yanelly Cruz Nicolás

Elixir y Erlang

Erlang es un lenguaje de programación funcional y concurrente desarrollado funcional y concurrente desarrollado originalmente por la empresa de telecomunicaciones Ericsson en la década de 1980. su diseño respondía a una necesidad estricta: crear sistemas de conmutación telefónica que nunca se cayeran y que soportan una disponibilidad del 99.999 %.

Su base. Aunque es un lenguaje independiente con una sintaxis moderna, limpia y muy expresiva (fuertemente inspirada en Ruby). Elixir corre sobre la máquina virtual BEAM, exactamente igual que Erlang.

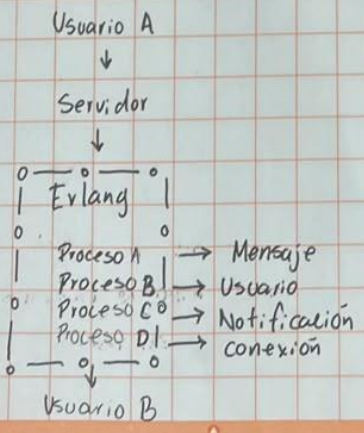


¿Dónde se usa en WhatsApp?.

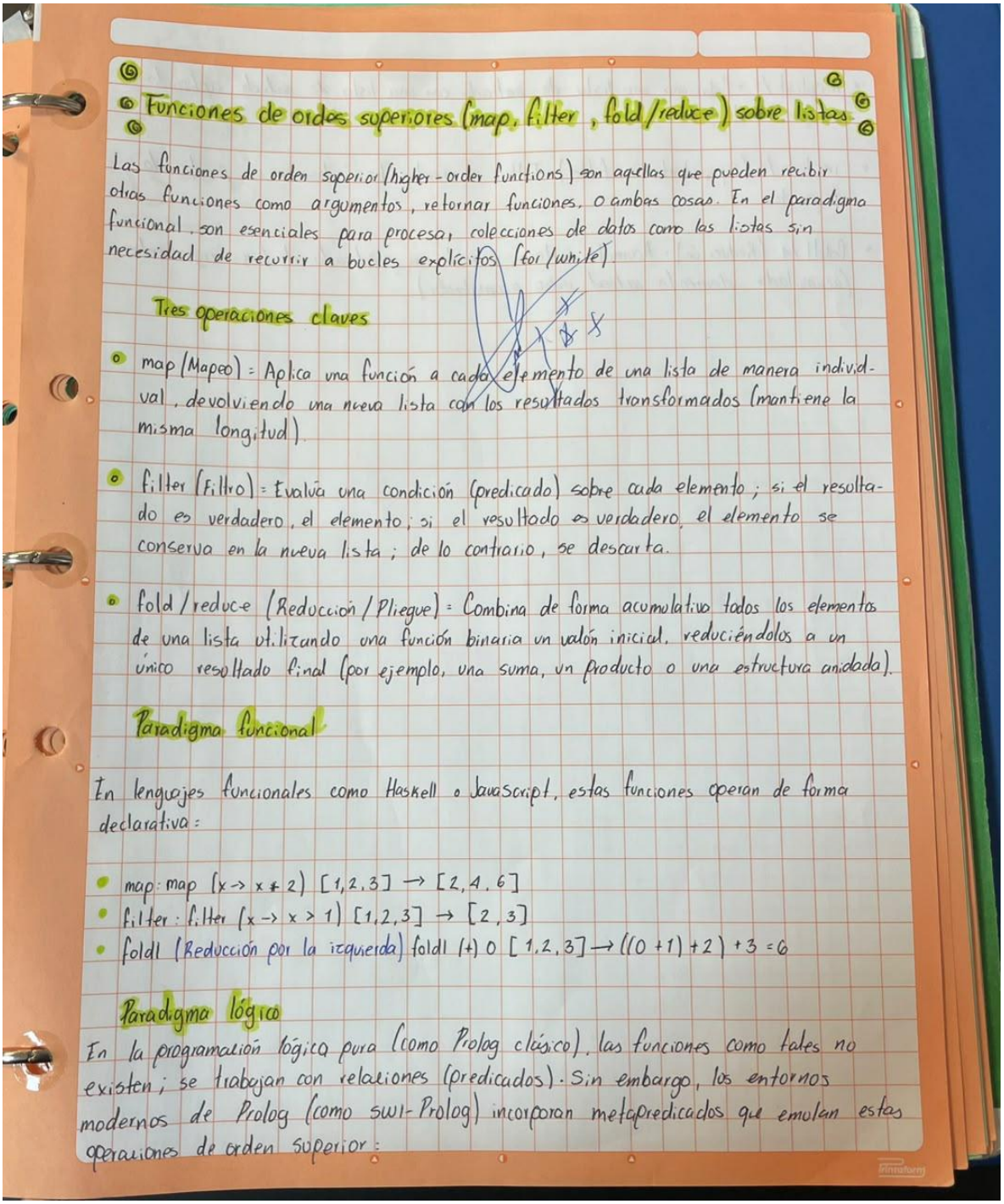
En WhatsApp, Elixir se ha utilizado principalmente en el backend, especialmente en sistemas que necesitan manejar muchísimas conexiones simultáneas.

Backend: Es la parte de una aplicación que funciona detrás de lo que se ve el usuario. Se encarga de procesar la información, aplicar las reglas del sistema y comunicarse con la base de datos.

WhatsApp es un ejemplo famoso de tecnología relacionada con Erlang. Su infraestructura históricamente se construyó utilizado Erlang porque permite manejar una enorme cantidad de conexiones y procesos simultáneos



Nombre: Yanelly Cruz Nicolás



Funciones de ordes superiores (map, filter, fold/reduce) sobre listas

Las funciones de orden superior (higher-order functions) son aquellas que pueden recibir otras funciones como argumentos, retornar funciones, o ambas cosas. En el paradigma funcional, son esenciales para procesar colecciones de datos como las listas sin necesidad de recurrir a bucles explícitos (for/while).

Tres operaciones claves

- map (Mapeo) = Aplica una función a cada elemento de una lista de manera individual, devolviendo una nueva lista con los resultados transformados (mantiene la misma longitud).
- filter (Filtro) = Evalúa una condición (predicado) sobre cada elemento; si el resultado es verdadero, el elemento; si el resultado es verdadero, el elemento se conserva en la nueva lista; de lo contrario, se descarta.
- fold/reduce (Reducción/Pliegue) = Combina de forma acumulativa todos los elementos de una lista utilizando una función binaria un valón inicial, reduciéndolos a un único resultado final (por ejemplo, una suma, un producto o una estructura anidada).

Paradigma funcional

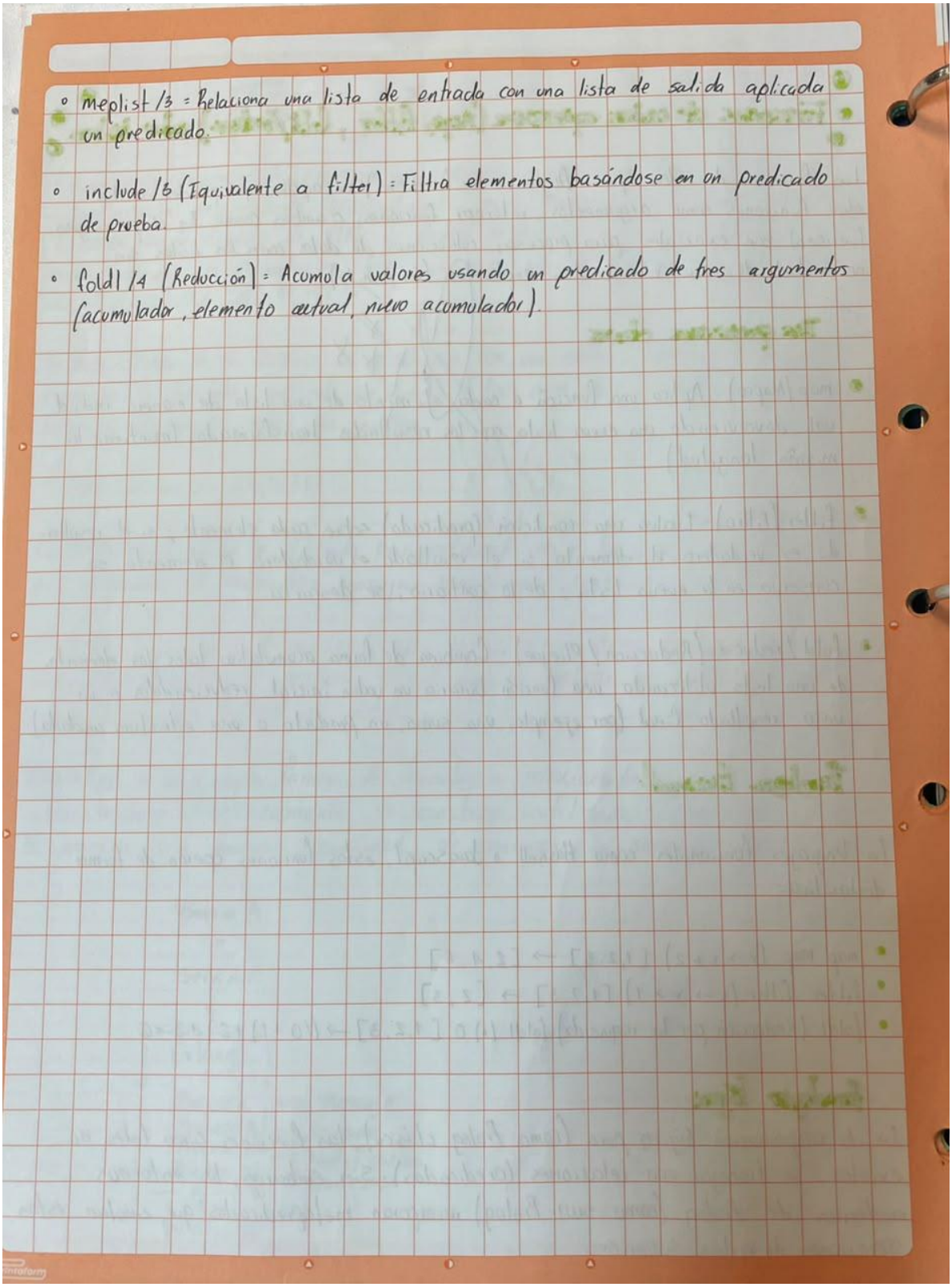
En lenguajes funcionales como Haskell o JavaScript, estas funciones operan de forma declarativa:

- map: map (x -> x * 2) [1,2,3] → [2,4,6]
- filter: filter (x -> x > 1) [1,2,3] → [2,3]
- foldl (Reducción por la izquierda) foldl (+) 0 [1,2,3] → ((0 + 1) + 2) + 3 = 6

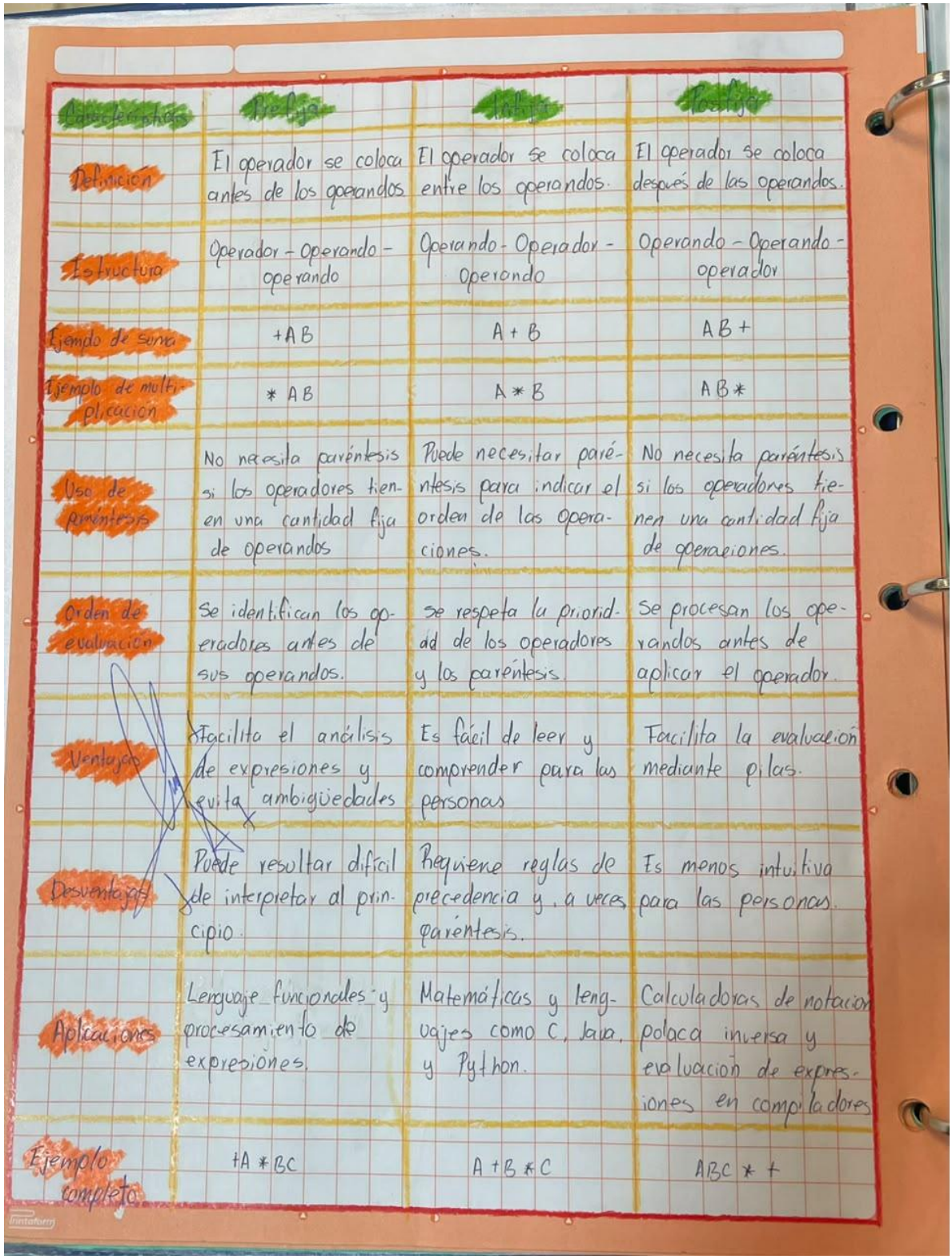
Paradigma lógico

En la programación lógica pura (como Prolog clásico), las funciones como tales no existen; se trabajan con relaciones (predicados). Sin embargo, los entornos modernos de Prolog (como swi-Prolog) incorporan metapredicados que emulan estas operaciones de orden superior:

Nombre: Yanelly Cruz Nicolás

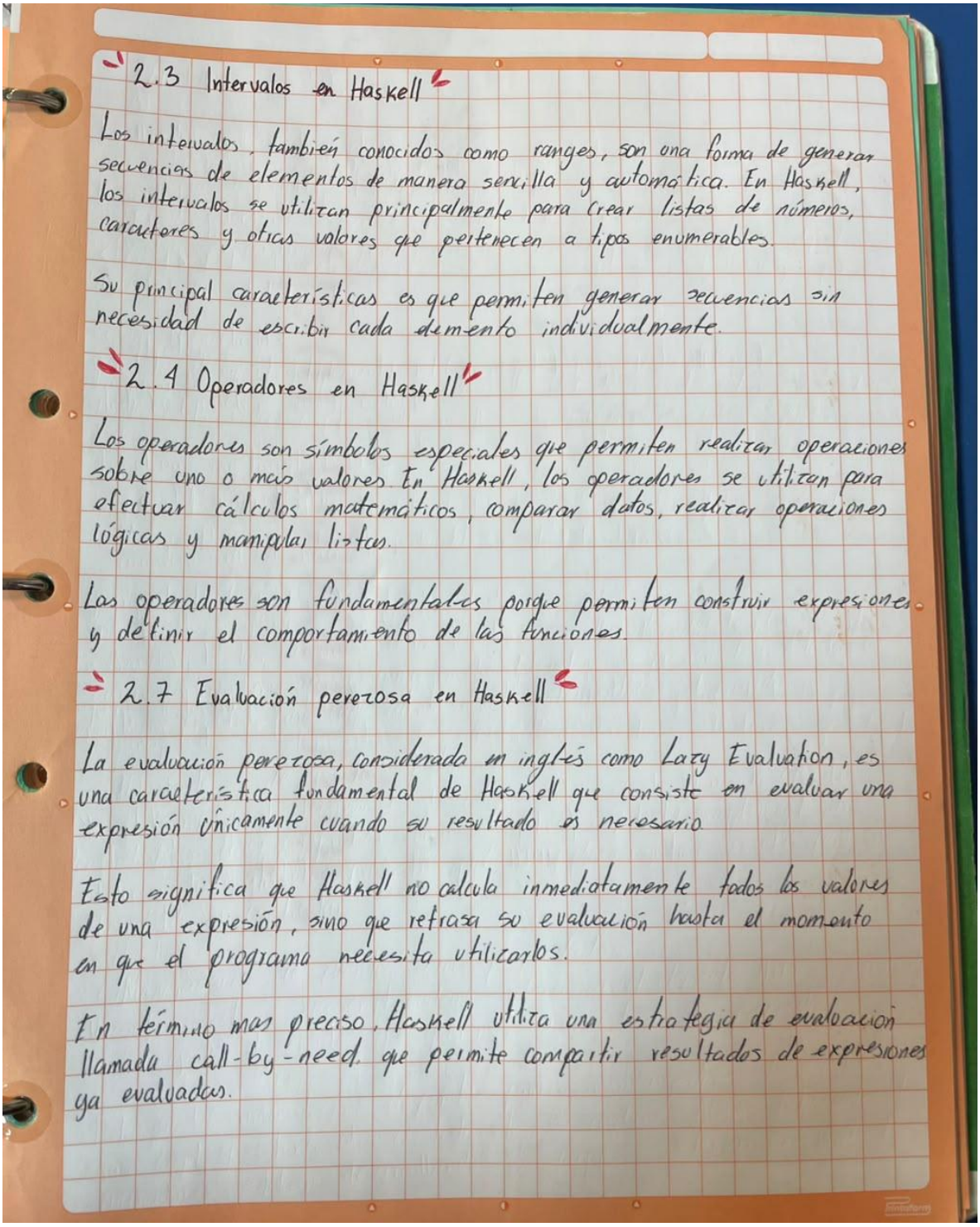


Nombre: Yanelly Cruz Nicolás



Características	Prefija	Infija	Postfija
Definición	El operador se coloca antes de los operandos	El operador se coloca entre los operandos.	El operador se coloca después de los operandos.
Estructura	Operador - Operando - operando	Operando - Operador - Operando	Operando - Operando - operador
Ejemplo de suma	+AB	A + B	AB +
Ejemplo de multiplicación	* AB	A * B	AB *
Uso de paréntesis	No necesita paréntesis si los operadores tienen una cantidad fija de operandos	Puede necesitar paréntesis para indicar el orden de las operaciones.	No necesita paréntesis si los operadores tienen una cantidad fija de operaciones.
Orden de evaluación	Se identifican los operadores antes de sus operandos.	se respeta la prioridad de los operadores y los paréntesis	Se procesan los operandos antes de aplicar el operador.
Ventajas	Facilita el análisis de expresiones y evita ambigüedades	Es fácil de leer y comprender para las personas	Facilita la evaluación mediante pilas.
Desventajas	Puede resultar difícil de interpretar al principio.	Requiere reglas de precedencia y, a veces, paréntesis.	Es menos intuitiva para las personas.
Aplicaciones	Lenguaje funcionales y procesamiento de expresiones.	Matemáticas y lenguajes como C, Java, y Python.	Calculadoras de notación polaca inversa y evaluación de expresiones en compiladores
Ejemplo completo	+A * BC	A + B * C	ABC * +

Nombre: Yanelly Cruz Nicolás



2.3 Intervalos en Haskell

Los intervalos, también conocidos como ranges, son una forma de generar secuencias de elementos de manera sencilla y automática. En Haskell, los intervalos se utilizan principalmente para crear listas de números, caracteres y otros valores que pertenecen a tipos enumerables.

Su principal características es que permiten generar secuencias sin necesidad de escribir cada elemento individualmente.

2.4 Operadores en Haskell

Los operadores son símbolos especiales que permiten realizar operaciones sobre uno o más valores. En Haskell, los operadores se utilizan para efectuar cálculos matemáticos, comparar datos, realizar operaciones lógicas y manipular listas.

Los operadores son fundamentales porque permiten construir expresiones y definir el comportamiento de las funciones.

2.7 Evaluación perezosa en Haskell

La evaluación perezosa, considerada en inglés como Lazy Evaluation, es una característica fundamental de Haskell que consiste en evaluar una expresión únicamente cuando su resultado es necesario.

Esto significa que Haskell no calcula inmediatamente todos los valores de una expresión, sino que retrasa su evaluación hasta el momento en que el programa necesita utilizarlos.

En término mas preciso, Haskell utiliza una estrategia de evaluación llamada call-by-need que permite compartir resultados de expresiones ya evaluadas.

Nombre: Yanelly Cruz Nicolás